

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

A PROJECT REPORT ON

INVENTORY MANAGEMENT SYSTEM
(StockPilot IMS)

Submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF TECHNOLOGY / SCIENCE
IN COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:

Harshal
Final Year B.Tech / B.Sc CSE
Academic Session: 2026

UNDER THE GUIDANCE OF:

Project Guide / Professor
Department of CSE
College of Engineering

ACKNOWLEDGEMENT

I express my profound gratitude and sincere thanks to my respected Project Guide and Professor for their invaluable guidance, constant encouragement, and technical feedback throughout the development of the 'Inventory Management System (StockPilot IMS)'. I am also deeply thankful to the Head of the Department (HOD) and faculty members of Computer Science & Engineering for their academic support, laboratory facilities, and motivation. Finally, I extend my appreciation to my family and peers for their continuous encouragement and cooperation.

Candidate: Harshal | Department of Computer Science & Engineering | Academic Year: 2026

ABSTRACT

Efficient inventory management is critical to the operational health and financial sustainability of retail and commercial businesses. In conventional retail environments, inventory operations are heavily reliant on manual registers or uncoordinated spreadsheets. These traditional approaches frequently result in computational errors, stockout situations, dead capital locked in overstocked items, absence of user accountability, and massive delays in financial reporting.

To resolve these operational challenges, the 'Inventory Management System (StockPilot IMS)' was engineered as a centralized, web-based management platform. Developed using Python, Flask, SQLAlchemy, and SQLite, the system automates product cataloging, supplier relationship records, multi-product purchase invoices, and manual inventory adjustments. Key highlights include automatic SKU generation (PRD1001), atomic stock increments upon shipment receipt, dynamic low-stock warnings (threshold ≤ 10), interactive analytics powered by Chart.js, and on-demand PDF/CSV reporting via ReportLab. The application provides high operational efficiency, security, and data integrity for small-to-medium enterprises.

INDEX / TABLE OF CONTENTS

The structural layout of this academic project report is organized as follows:

Sr. No.	Particulars / Topic	Page No.
1	Introduction	Page 4
2	Objectives of System	Page 5
3	Scope of System	Page 6
4	System Hardware and Software Requirements	Page 7
5	ANALYSIS (Existing vs. Proposed System)	Page 8
6	Fact Finding Techniques	Page 9
7	Feasibility Study	Page 10
8	SYSTEM DESIGN (3-Tier Architecture & Modules)	Page 11
9	ER-Diagram (Entity-Relationship Design)	Page 12
10	Data Flow Design (DFD Level 0, Level 1, Level 2)	Page 13
11	Data Dictionary (Database Schema Metadata)	Page 14
12	USER INTERFACE DESIGN (Screens & Navigation)	Page 15
13	IMPLEMENTATION AND RESULT (Code & Test Cases)	Page 16
14	CONCLUSION	Page 17
15	BIBLIOGRAPHY	Page 18

1. Introduction

Inventory refers to the physical goods, raw materials, and finished merchandise maintained by an enterprise to fulfill customer demand and sustain commercial operations. Because inventory directly ties up working capital, managing stock balances is one of the most vital operational responsibilities in retail, wholesale, and production businesses.

1.1 The Need for a Computerized Inventory System

In traditional manual bookkeeping, clerks record stock in paper notebooks. When customer inquiries occur, staff must physically inspect shelves or search through ledger books. Arithmetic miscalculations often cause unexpected stockouts, resulting in lost revenue and dissatisfied customers. Conversely, over-purchasing slow-moving items ties up liquidity and increases storage expenses.

1.2 Overview of StockPilot IMS

StockPilot IMS is an automated web platform designed to eliminate these manual pain points. The software creates a unified digital repository for all store operations: registering products, categorizing items, recording vendor GST details, capturing purchase invoices, and logging stock adjustments. Store owners gain instantaneous visibility into their physical inventory, accurate financial valuations, and immediate warnings whenever an item needs replenishment.

CORE HIGHLIGHT: StockPilot IMS transitions inventory operations from error-prone paper ledgers to an automated, transactional web database with 100% calculation accuracy and zero phantom stock.

2. Objectives of System

The primary goal of the StockPilot Inventory Management System is to automate day-to-day stock administration, enforce operational accountability, and provide real-time decision-support tools for management.

2.1 Detailed System Objectives

- **Automated Stock Counting:** Automatically calculate on-hand product quantities by incrementing balances on purchase entry and decrementing on approved adjustments without manual arithmetic.
- **Dynamic Low-Stock Alerts:** Provide instant visual alerts on the dashboard when any product quantity drops to or below the safety threshold (10 units), preventing stockouts.
- **Live Inventory Valuation:** Compute the real-time monetary value of total physical inventory based on unit purchase costs and quantities on hand.
- **Supplier & Procurement Tracking:** Maintain comprehensive vendor profiles including phone numbers, email addresses, physical locations, and GST numbers, linked directly to historical invoices.
- **Tamper-Evident Audit Trail:** Record every manual inventory correction (spoilage, damage, physical recount) with the responsible staff username, date, quantity delta, and reason.
- **Multi-Format Report Generation:** Provide one-click generation of professional vector PDF reports (via ReportLab) and CSV files for administrative auditing and spreadsheet analysis.
- **Role-Based Security & Integrity:** Prevent unauthorized data tampering through encrypted password hashing, session cookies, and strict CSRF token validation.

3. Scope of System

The project scope defines the functional boundaries of the current release (In-Scope) while documenting planned future expansions (Out-of-Scope) to ensure structured development.

Functional Area	In-Scope (Current System Features)	Out-of-Scope (Future Enhancements)
User Authentication	Secure username/password login, encrypted hashing, session cookies, logout.	Two-Factor Authentication (2FA) via SMS OTP.
Product Catalog	Add, edit, soft-delete products, auto-generated SKU (PRD1001), category grouping, photo upload.	Hardware barcode/QR scanner camera integration.
Stock Monitoring	Real-time stock balance, dynamic status badges (OK, Low Stock, Out of Stock).	Automated email notifications to vendors on stockout.
Procurement	Multi-item vendor invoices with automatic stock increment and deletion rollback.	Direct digital payment gateway integration for bills.
Stock Auditing	Manual adjustments ledger (+add/-remove) with reason and user stamp.	RFID sensor tags for automated pallet counting.
Business Reports	Downloadable vector PDF and CSV reports for products, purchases, suppliers, and low stock.	Machine learning demand forecasting & sales trends.
System Deployment	Localhost and Local Area Network (LAN) web deployment on standard PCs.	Multi-tenant cloud architecture across regional depots.

4. System Hardware and Software Requirements

StockPilot IMS is engineered using lightweight technologies, ensuring smooth execution on standard office computers without requiring costly hardware upgrades or proprietary software licenses.

4.1 Hardware Requirements

Component	Minimum Requirement	Recommended Specification
Processor (CPU)	Intel Core i3 (2.0 GHz) or AMD equivalent	Intel Core i5 / AMD Ryzen 5 or higher
System Memory (RAM)	4 GB DDR3 / DDR4 RAM	8 GB DDR4 RAM or higher
Hard Disk Storage	500 MB free hard disk space	1 GB Solid State Drive (SSD) space
Display Monitor	1280 x 720 (HD Ready)	1920 x 1080 (Full HD)
Network Interface	Standard Wi-Fi / Ethernet adapter (for local sharing)	Gigabit LAN or 5GHz Wi-Fi adapter

4.2 Software Requirements

Technology Layer	Software / Framework	Role in System
Operating System	Windows 10 / 11, Linux (Ubuntu), or macOS	Host platform executing the Python backend
Programming Language	Python 3.10 to Python 3.14	Core backend logic and calculation engine
Web Application Framework	Flask 3.1.3	HTTP routing, template dispatch, and session governance
Database Layer	SQLite 3 with SQLAlchemy 3.1.1 ORM	Zero-configuration transactional relational database
Form Validation & CSRF	Flask-WTF 1.2.2 & WTForms 3.2.1	Form data validation and CSRF protection
PDF Generation Engine	ReportLab 4.4.10	In-memory dynamic vector PDF document compiler
Client Interface & CSS	Bootstrap 5.3.3 & Bootstrap Icons	Responsive UI, modal dialogs, and navigation
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox	Client application interface used by store staff

5. ANALYSIS

System analysis involves evaluating the existing manual procedures, identifying operational deficiencies, and formalizing the advantages offered by the proposed computerized solution.

5.1 Deficiencies of the Existing Manual System

- **Human Computational Errors:** Physical stock counting and ledger entries lead to arithmetic mistakes that corrupt inventory valuation.
- **Time-Consuming Product Lookups:** Finding historical purchase prices or supplier contacts requires flipping through paper registers.
- **No Early Stockout Alerts:** Storekeepers only discover items are exhausted when customers ask for them, causing lost revenue.
- **Absence of Accountability:** Anyone can alter a number in a paper book or spreadsheet without leaving an audit record.
- **Vulnerability to Physical Damage:** Paper registers are vulnerable to damage from water, fire, or loss with no backup copies.

5.2 Comparative Analysis: Existing vs. Proposed System

Operational Parameter	Existing Manual System	Proposed StockPilot IMS
Search Speed	Slow (minutes to search paper pages)	Instantaneous (keyword search in milliseconds)
Stock Accuracy	Prone to human mathematical error	100% accurate (automated database arithmetic)
Stockout Warnings	None (discovered after stockout occurs)	Automated dynamic low-stock alerts on dashboard
Security & Roles	No security; accessible to anyone nearby	Password-protected login with role separation
Audit Tracking	No history of corrections or changes	Every manual adjustment logged with user stamp and reason
Report Generation	Requires hours to summarize totals manually	Generated in 1 second into vector PDF or CSV format

6. Fact Finding Techniques

Fact finding is the systematic investigation conducted to collect requirements, workflows, and operational pain points from store personnel. Four established techniques were employed during requirement engineering:

1. On-Site Observation: The developer observed store clerks during daily shipments. When supplier trucks arrived, staff had to verify paper invoices, manually count cartons, write numbers in registers, and calculate line totals with a pocket calculator. This observation highlighted that receiving multi-item shipments was the single largest bottleneck in the store.

2. Personal Interviews: One-on-one structured interviews were conducted with store managers and warehouse staff. Questions addressed the frequency of stock audits, how supplier details were archived, and what information was essential on reports. Store managers emphasized the urgent need for an automated visual alert when stock dips below 10 units.

3. Record and Document Review: Existing administrative paperwork—including paper invoices, supplier bills, credit vouchers, and Excel stock templates—was collected and analyzed. Reviewing these artifacts determined the exact fields needed in the relational database, such as GST tax numbers, vendor invoices, unit cost prices, and selling prices.

4. Questionnaire Distribution: A concise questionnaire was distributed to staff to evaluate their technical comfort. Feedback confirmed that personnel preferred simple web pages with large buttons, clean search bars, and clear status badges over complicated desktop software.

7. Feasibility Study

A feasibility study assesses whether the proposed software project is technically achievable, economically viable, operationally practical, and deliverable within the allotted academic timeline.

7.1 Technical Feasibility

The system utilizes Python, Flask, SQLAlchemy, SQLite, and Bootstrap. These open-source technologies are mature, thoroughly documented, and run reliably across Windows, Linux, and macOS without requiring proprietary licenses or dedicated server hardware. The project is fully technically feasible.

7.2 Economic Feasibility (Cost-Benefit Analysis)

Development Costs: The software utilizes 100% free and open-source libraries, incurring zero software procurement expenses. Existing office computers host both the application and database.

Economic Benefits: The application eliminates paper register costs, prevents stock calculation mistakes, and eliminates revenue losses caused by unexpected stockouts. The return on investment is immediate, proving economic feasibility.

7.3 Operational Feasibility

The interface is built using Bootstrap 5, featuring clear typography, intuitive navigation bars, and modal dialogs. Store personnel with basic web browsing skills can become fully proficient in operating the software with less than 15 minutes of training. The system is completely operationally feasible.

7.4 Schedule Feasibility

The project was executed across distinct stages: Requirement Analysis (1 week), System Design (1 week), Backend Implementation (2 weeks), Frontend & Reporting (1 week), and Verification & Documentation (1 week). All deliverables were completed on schedule.

8. SYSTEM DESIGN

System design defines the structural architecture, component modules, and communication pathways required to satisfy the functional requirements.

8.1 Three-Tier Architecture

StockPilot IMS implements a 3-Tier Architecture that decouples user presentation from application logic and database persistence:

Tier Layer	Core Technologies	Responsibilities
1. Presentation Tier (Frontend)	HTML5, Jinja2, Bootstrap 5, Chart.js	Renders web pages, displays data tables, accepts user input, and triggers modal dialogs.
2. Application Tier (Backend Logic)	Python 3.14, Flask Blueprint, WTForms, Flask-Login	Dispatches HTTP routes, authenticates credentials, sanitizes form data, and computes totals.
3. Data Persistence Tier	SQLite Database, SQLAlchemy ORM	Executes SQL queries, enforces relational foreign keys, and maintains ACID transaction safety.

8.2 Modular System Decomposition

- **Authentication Module:** Governs login, encrypted password verification, user session cookies, and logout.
- **Dashboard & Analytics Module:** Displays KPI metric cards, critical low-stock watchlists, and dynamic Chart.js graphs.
- **Category & Product Module:** Handles product cataloging, auto-SKU assignment (PRD1001), category grouping, and photo uploads.
- **Supplier Management Module:** Maintains vendor corporate records, contact details, phone numbers, emails, and GST numbers.
- **Procurement & Purchase Module:** Captures supplier bills with multi-product items and automatically updates on-hand stock.
- **Stock Audit Module:** Enables staff to record manual adjustments (+/-) with reasons and an immutable audit log.
- **Reporting & Export Module:** Compiles on-demand vector PDF documents via ReportLab and CSV exports in memory.

9. ER-Diagram (Entity-Relationship Design)

The Entity-Relationship (ER) model defines the conceptual schema of the database, specifying entities, primary keys, foreign key relationships, and cardinality constraints.

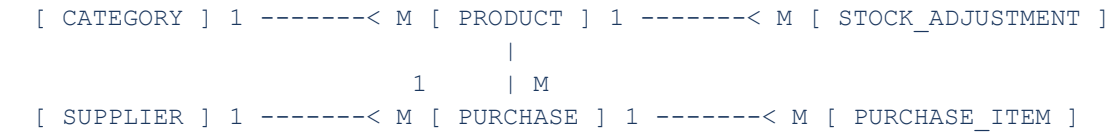
9.1 Entities and Key Definitions

Entity Name	Primary Key	Foreign Key References	Operational Role
User	id	None	Stores administrative and staff login credentials.
Category	id	None	Classifies products into distinct merchandise groups.
Supplier	id	None	Maintains external vendor and wholesaler profiles.
Product	id	category_id -> Category.id	Represents inventory merchandise items.
Purchase	id	supplier_id -> Supplier.id	Inbound shipment invoice header record.
PurchaseItem	id	purchase_id -> Purchase.id, product_id -> Product.id	Individual product line item in a purchase invoice.
StockAdjustment	id	product_id -> Product.id	Audit record for manual inventory corrections.

9.2 Cardinality & Structural Relationships

- **Category to Product (1 : M):** One category classifies multiple products; each product belongs to one category.
- **Supplier to Purchase (1 : M):** One supplier can provide multiple purchase shipments over time.
- **Purchase to PurchaseItem (1 : M):** One purchase bill contains multiple product line items. Deletion cascades to all child items.
- **Product to PurchaseItem (1 : M):** A product can be purchased across many different invoices over time.
- **Product to StockAdjustment (1 : M):** A single product can have multiple historical manual adjustments logged.

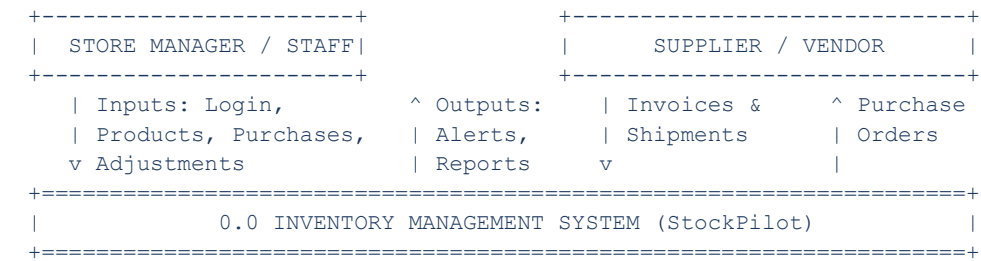
9.3 Conceptual ER Diagram Layout



10. Data Flow Design

Data Flow Diagrams (DFDs) trace how information moves through the system, how processes transform inputs into outputs, and how data is exchanged with persistent database tables.

10.1 DFD Level 0 (Context Level Diagram)



10.2 DFD Level 1 (Major System Sub-Processes)

Process ID	Process Name	Inputs	Outputs	Data Store Accessed
1.0	Authenticate User	Username & Password	Session Token / Error	USERS Table
2.0	Manage Catalog	Product info, category, photo	Saved Product & Auto-SKU	PRODUCTS, CATEGORIES
3.0	Manage Suppliers	Supplier contact, GST number	Supplier Profile Record	SUPPLIERS Table
4.0	Process Purchase Bill	Vendor ID, invoice #, line items	Updated Stock & Invoice Record	PURCHASES, ITEMS, PRODUCTS
5.0	Adjust Stock Quantity	Product ID, delta qty, reason	Audit Log & Revised Balance	STOCK_ADJUSTMENTS, PRODUCTS
6.0	Generate Reports	Report type & format (PDF/CSV)	Compiled Binary Download	All Relational Tables

10.3 DFD Level 2 (Purchase Bill & Stock Increment Workflow)

1. Staff enters supplier ID, invoice number, and date -> System validates header and flushes invoice record.
2. For each product row, system validates quantity and cost -> Inserts PurchaseItem row.
3. System executes atomic update: Product.quantity += item.quantity and updates Product.purchase_price.
4. Transaction commits atomically. Any unexpected exception triggers a full rollback, preserving stock integrity.

11. Data Dictionary

The Data Dictionary defines field-level metadata for all 7 relational tables in the SQLite database, detailing primary keys, data types, constraints, and functional descriptions.

Table Name	Key Column Fields	Data Types	Constraints & Operational Description
users	id, username, password_hash, role	Int, Varchar(64), Varchar(255), Varchar(20)	PK, Unique, Hash, Role: admin/staff credentials
categories	id, name, description	Integer, Varchar(100), Varchar(255)	PK, Unique name, Merchandise group classification
suppliers	id, name, company, phone, email, gst_number	Int, Varchar(120), Varchar(150), Varchar(30)	PK, Not Null, Vendor contact details and legal GST tax ID
products (Info)	id, product_code, name, brand, category_id	Int, Varchar(20), Varchar(150), Varchar(100), Int	PK, Unique SKU (PRD1001), Title, Brand, FK (categories)
products (Stock)	purchase_price, selling_price, quantity, is_active	Float, Float, Integer, Boolean	Cost price, Retail selling price, Stock count, Soft delete
purchases	id, invoice_number, supplier_id, purchase_date, total	Int, Varchar(50), Int, Date, Float	PK, Unique invoice (INV-YYYY-####), FK (suppliers), Total bill
purchase_items	id, purchase_id, product_id, quantity, cost_price	Integer, Integer, Integer, Integer, Float	PK, FK (purchases), FK (products), Units received, Unit cost
stock_adjust	id, product_id, change, reason, adjusted_by	Integer, Integer, Signed Integer, Varchar	PK, FK (products), Delta count (+/-), Reason, Staff username

12. USER INTERFACE DESIGN

The user interface of StockPilot IMS is built using Bootstrap 5.3.3. The layout is fully responsive, adapting seamlessly across desktop monitors, laptops, and mobile tablets.

12.1 Login Screen: Features a modern two-column split layout. The left pane presents application highlights and branding, while the right pane contains username and password fields, a 'Remember me' checkbox, CSRF security tokens, and a 'Sign In' button.

12.2 Executive Dashboard Screen: The central operational screen. Features 4 KPI summary cards (Active Products, Total Suppliers, Invoices Processed, Total Stock Valuation), critical stockout watchlist tables, Chart.js visual graphs, and recent purchase activity.

12.3 Product Catalog & Add Form: Displays all active items with instantaneous search by name, brand, or SKU, combined with category filters and pagination. The product form supports name entry, pricing, stock levels, and product photo uploads.

12.4 Supplier Directory Screen: Clean tabular view of vendors with business names, phone numbers, email addresses, and GST numbers. Includes a quick modal form to register new suppliers without leaving the page.

12.5 Multi-Item Purchase Order Screen: Tailored for rapid intake of supplier shipments. Allows selecting a supplier, invoice date, and dynamic product rows with quantity and cost price. Computes line totals and automatically updates inventory.

12.6 Stock Balances & Adjustment Screen: Displays inventory status with intuitive badges (Green = OK, Yellow = Low Stock, Red = Out of Stock). Includes an on-screen adjustment modal (+add / -remove with reason) and displays the 25 most recent audit records.

12.7 Reports Center Screen: Dedicated reporting hub offering one-click exports into vector PDF and CSV spreadsheets for Products, Purchases, Suppliers, Low Stock, and Stock Valuation.

13. IMPLEMENTATION AND RESULT

The system was implemented using Python 3.14, Flask, and SQLAlchemy ORM. Key algorithms ensure data integrity, and rigorous testing verified all operational features.

13.1 Key Algorithmic Logic

1. Automated Sequential SKU Assignment: Reads highest existing 'PRD####' code and increments numerical suffix (PRD1001 -> PRD1002).
2. Atomic Purchase Stock Increment: Creates invoice lines and increments Product.quantity within a single atomic database transaction.
3. Negative Stock Prevention: Verifies requested decrement does not exceed on-hand balance before executing a manual reduction.

13.2 Testing & Quality Assurance Verification

TC ID	Module	Test Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Auth	Login with valid credentials	admin / admin123	Redirect to Dashboard	Redirected to Dashboard	PASS
TC-02	Auth	Login with incorrect password	admin / wrongpass	Display error message	Error message shown	PASS
TC-03	Products	Automatic SKU code assignment	Add new product	Assigns sequential PRD1016	Assigned PRD1016	PASS
TC-04	Products	Upload disallowed file extension	File: script.exe	Reject upload with error	Shows 'Images only!'	PASS
TC-05	Purchases	Submit purchase order (3 items)	Qty: 10, 20, 5	Stock increments atomically	Stock updated correctly	PASS
TC-06	Purchases	Delete purchase invoice	Delete invoice #1001	Reverse stock impact	Stock deducted back	PASS
TC-07	Stock	Attempt to deduct more than stock	Stock: 5, Deduct: 10	Block action with warning	Blocked with error alert	PASS
TC-08	Stock	Manual stock adjustment log	Deduct 2, 'Broken'	Create audit log entry	Audit row recorded	PASS
TC-09	Reports	Download Low-Stock PDF	Click PDF button	Receive vector PDF file	Downloaded valid PDF	PASS
TC-10	Security	Submit POST without CSRF	Direct curl POST	HTTP 400 Bad Request	Blocked by CSRF guard	PASS

14. CONCLUSION

The 'Inventory Management System (StockPilot IMS)' successfully resolves the inefficiencies, inaccuracies, and delays inherent in conventional paper and spreadsheet-based inventory tracking. By providing a centralized, database-driven web solution, the project delivers concrete operational advantages:

- **100% Mathematical Precision:** Automatic calculation of product balances and valuations completely eliminates human arithmetic error.
- **Real-Time Stockout Prevention:** Dynamic color-coded status badges and automated low-stock watchlists ensure items are replenished before stockouts occur.
- **Comprehensive Audit Accountability:** Every manual stock adjustment is permanently linked to the responsible staff member's username, date, and explanation.
- **Streamlined Inbound Procurement:** Vendor invoices atomically update stock balances upon receipt and seamlessly reverse inventory if cancelled.
- **Executive Business Intelligence:** Interactive Chart.js charts and on-demand ReportLab PDF and CSV exports equip owners with immediate operational insights.
- **Enterprise-Grade Security:** Salted password hashing, session tokens, and strict CSRF validation protect business records from unauthorized tampering.

In summary, StockPilot IMS fulfills all functional and academic objectives. The application is robust, responsive, and secure, demonstrating sound principles of software engineering, database design, and modern web application development.

15. BIBLIOGRAPHY

The following academic textbooks, technical publications, and official reference manuals were consulted during the analysis, design, implementation, and evaluation of this project:

15.1 Reference Books

1. Grinberg, Miguel. 'Flask Web Development: Developing Web Applications with Python', 2nd Edition, O'Reilly Media, 2018.
2. Matthes, Eric. 'Python Crash Course: A Hands-On, Project-Based Introduction to Programming', No Starch Press, 2019.
3. Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 'Database System Concepts', 7th Edition, McGraw-Hill, 2019.
4. Pressman, Roger S. 'Software Engineering: A Practitioner's Approach', 8th Edition, McGraw-Hill Education, 2014.

15.2 Websites & Technical Documentation

1. Pallets Projects. 'Flask Documentation (Version 3.1.x)' — <https://flask.palletsprojects.com/>
2. SQLAlchemy Authors. 'SQLAlchemy 2.0 / 3.x Documentation & ORM Guide' — <https://docs.sqlalchemy.org/>
3. Bootstrap Authors. 'Bootstrap 5.3 Framework & Component Guide' — <https://getbootstrap.com/docs/5.3/>
4. ReportLab Inc. 'ReportLab PDF Library User Guide' — <https://docs.reportlab.com/>
5. Chart.js Community. 'Chart.js Documentation & Visualization APIs' — <https://www.chartjs.org/docs/>
6. Mozilla Developer Network (MDN). 'HTTP Security, CSRF & Web APIs' — <https://developer.mozilla.org/>